

基于Java生态的Web应用用户鉴权立体化防护架构设计

郑秀琴, 王飞平

(衢州职业技术学院, 浙江 衢州 324000)

摘要: 随着Web应用的普及, 用户鉴权安全成为保障系统数据安全的核心环节。传统单一的安全防护手段难以应对复杂的网络攻击。基于Java生态体系, 提出了一种包含网络层、认证层、授权层、会话层和监控层的立体防护架构, 详细阐述了各层的核心技术原理、实现方案及协同机制。该架构能够系统性地抵御常见安全威胁, 为Java Web应用提供全面的鉴权安全解决方案。

关键词: 用户鉴权; Web安全; 会话管理; Java语言

DOI:10.16184/j.cnki.comprg.2025.09.001

1 概述

在数字化转型背景下, Web应用已成为网络服务的主要载体。用户鉴权作为访问控制的第一道关卡, 其安全性直接影响系统的数据完整性、可用性和保密性。然而, 当前Web应用面临的安全威胁呈现多样化趋势, 包括身份令牌劫持、跨站请求伪造(CSRF)、权限越界、会话固定攻击等。传统单一的认证方式(例如, 用户名/密码)和简单的安全配置已难以应对复杂攻击场景, 亟须构建多层次、立体化的安全防护体系。

Java生态凭借其跨平台特性和丰富的安全框架(例如, Spring Security等), 为Web应用鉴权提供了强大的技术支持。立体化防护架构遵循“纵深防御”(Defense-in-Depth)原则, 在网络传输、身份认证、权限控制、会话管理、安全监控等不同层面设置独立的安全策略, 各层之间相互补充、协同防御, 形成完整的安全防护链条。研究结合Java技术栈, 系统性地设计并实现适用于Web应用的立体化鉴权防护架构, 旨在为开发者提供可落地的安全解决方案。

2 立体化防护架构总体设计

2.1 架构设计原则

分层隔离。将安全防护划分为独立的层次, 每层专注于特定的安全目标(例如, 网络层加密、认证层身份验证), 降低模块间耦合度。

渐进增强。底层防护为上层提供基础安全保障, 上层在此基础上实现更细粒度的安全策略(例如, 业务层的RBAC基于认证层的用户身份)。

标准兼容。遵循OAuth2、OpenID Connect、JWT等国际标准, 确保架构的通用性和互操作性。

动态响应。通过监控层实时收集安全事件, 动态调整各层防护策略(例如, 封禁恶意IP、限制登录频率等)。

2.2 立体化架构模型

研究提出的立体化架构包括5个层次, 如表1所示。

表1 立体化分层架构表

层次划分	安全防护
网络层	实现传输安全与边界防护, 包括HTTPS加密、安全头配置、反向代理过滤等
认证层	验证用户身份, 支持多种认证方式(例如, JWT、OAuth2、双因素认证)
授权层	基于用户身份和属性进行权限控制, 实现RBAC、ABAC等授权模型
会话层	管理用户会话生命周期, 确保令牌安全存储与传输
监控层	实时监控认证行为, 记录日志并实施攻击检测

3 立体防护架构的详细设计

3.1 网络层防护——传输安全与边界过滤

网络层是防护架构的最外层, 负责过滤恶意请求、加密数据传输, 并为后续层次提供安全的通信环境。

3.1.1 HTTPS强制实施

3.1.1.1 技术原理

通过TLS/SSL协议对客户端与服务器之间的通信进行加密, 防止数据在传输过程中被窃听或篡改。需配置强加密套件(例如, TLS 1.3), 禁用弱算法(例如, RC4、MD5), 并使用证书透明度(Certificate Transparency)机制防止证书伪造。

3.1.1.2 Java生态实现

Tomcat 配置如下。

```
<Connector port="443" protocol="HTTP/1.1" SSLEnabled="true"
    maxThreads="150" scheme="https" secure="true"
    clientAuth="false" sslProtocol="TLS"
    keystoreFile="conf/keystore.jks" keystorePass=
"YOURpwd!23"/>
```

在server.xml中添加HTTPS连接器, SpringBoot自动配置, 通过application.properties启用HTTPS, 代码如下。

作者简介: 郑秀琴(1968—), 女, 高级工程师, 副教授, 研究方向为计算机网络与网络安全技术。

```
server.ssl.key-store=classpath:keystore.jks server.
ssl.key-store-password=YOURpwd! 23 server.ssl.
key-store-type=JKS server.port=443
```

3.1.2 安全响应头配置

3.1.2.1 核心安全头

Content-Security-Policy (CSP)。限制资源加载来源，防御 XSS 攻击，例如，Content-Security-Policy: default-src 'self'; script-src 'self' https://trusted.XXX.com。

Strict-Transport-Security (HSTS)。强制浏览器使用 HTTPS 通信，例如，Strict-Transport-Security: max-age=31536000; includeSubDomains。

X-Content-Type-Options。禁用 MIME 类型嗅探，防止恶意文件执行，例如，X-Content-Type-Options: nosniff。

3.1.2.2 Spring Security 集成

通过 HttpSecurity 配置安全头，代码如下。

```
http.headers(headers -> headers
    .contentSecurityPolicy("default-src 'self'")
    .strictTransportSecurity(sts -> sts
        .maxAgeInSeconds(31536000)
        .includeSubDomains(true)
    )
    .xssProtection(xss -> xss.enabled(true)) );
```

3.1.3 反向代理与负载均衡

使用 Nginx/Apache 作为反向代理，实现以下功能。

3.1.3.1 请求速率限制

在 Nginx 上可以通过 limit_req 模块限制单个 IP 的请求频率，防止暴力破解。

3.1.3.2 恶意 IP 封禁

结合 Fail2ban 工具，自动封禁多次认证失败的 IP 地址。

3.2 认证层——多维度身份验证体系

认证层是用户进入系统的核心入口，负责验证用户声称的身份。Java 生态提供了丰富的认证框架，支持从简单密码认证到复杂的分布式令牌机制认证。

3.2.1 基础认证方式

3.2.1.1 表单认证与 Basic Auth

Spring Security 默认支持表单认证和 HTTP Basic 认证，通过配置 SecurityFilterChain 定义认证规则。

3.2.1.2 密码安全增强

(1) 哈希加密。使用 BCrypt、Argon2 等强哈希算法存储密码，避免明文传输，代码如下。

```
@Bean
public PasswordEncoder passwordEncoder() {
```

```
return new BCryptPasswordEncoder();
}
```

(2) 密码复杂度校验。通过正则表达式或 Hibernate Validator 强制密码策略，代码如下。

```
@Pattern(regexp = "(?=.[A-Z])(?=.[a-z])(?=.*\\d).{8,}$", message = "密码需包含大小写字母和数字，长度至少8位")
private String password;
```

3.2.2 JWT 无状态认证

3.2.2.1 JWT 原理与优势

JWT (JSON Web Token) 由 Header、Payload、Signature 三部分组成，通过数字签名保证令牌的完整性。无状态设计避免了服务器端的会话存储，适用于分布式微服务架构。

3.2.2.2 核心实现代码

令牌生成与验证 (使用 JWT 库)，自定义认证过滤器。

第三方认证集成 (OAuth2/OpenID Connect)。

通过 Spring Security OAuth2 Client 支持微信、钉钉等第三方登录，步骤如下。

第一步，在微信或钉钉开放平台注册应用，获取 appId 和 appSecret。

第二步，在 SpringBoot 项目里添加 Spring Security 和 Spring Security OAuth2 Client 依赖。

第三步，在 application.properties 或 application.yml 里配置微信客户端信息。

第四步，借助 Spring Security 提供的登录端点实现微信登录。

3.3 授权层——细粒度权限控制体系

授权层在认证通过的基础上，决定用户是否有权执行特定操作或访问资源，是分层架构中实现业务逻辑安全的关键环节。

3.3.1 基于角色的访问控制 (RBAC)

3.3.1.1 核心概念

RBAC 通过“用户 - 角色 - 权限”三元组模型，将权限分配给角色而非直接分配给用户，简化权限管理。常见角色包括 ROLE_ADMIN、ROLE_USER、ROLE_VIEWER 等。

3.3.1.2 Spring Security 实现

(1) 角色加载。从数据库获取用户角色。

(2) 接口级权限控制。通过 @PreAuthorize 注解限制访问。

3.3.2 基于属性的访问控制 (ABAC)

动态权限评估。ABAC 基于用户属性 (例如，部

门、职位)、资源属性(例如,数据敏感度)和环境属性(例如,访问时间、IP地址)进行权限决策,适用于复杂业务场景。

自定义权限评估器。定义实现Spring Security的PermissionEvaluator接口的CustomPermissionEvaluator类。

在配置类中注册评估器,代码如下。

```
@Bean
public PermissionEvaluator permissionEvaluator() {
    return new CustomPermissionEvaluator();
}
```

方法级安全与资源隔离。通过Spring Security的方法安全注解(@PreAuthorize、@PostAuthorize)实现细粒度控制。

3.4 会话层——安全的令牌与会话管理

会话层负责管理用户会话的创建、更新和销毁,确保令牌在生命周期内的安全性。

3.4.1 无状态会话与JWT令牌

3.4.1.1 无状态设计优势

JWT令牌包含用户身份和权限信息,服务器无需存储会话状态,支持水平扩展和微服务架构。令牌通过HTTP Header传输(例如,Authorization: Bearer <token>),避免Cookie被劫持的风险。

3.4.1.2 令牌生命周期管理

(1) 短期有效令牌。访问令牌(AccessToken)设置较短有效期(例如,30 min),降低被劫持后的风险。

(2) 刷新令牌(RefreshToken)。用于生成新的访问令牌,存储在安全的HttpOnly Cookie中,防止XSS攻击。

3.4.2 传统会话管理(Cookie-Session模式)

在需要有状态会话的场景(例如,表单提交)中,使用HttpOnly和Secure属性的Cookie。

HttpOnly。防止XSS脚本读取Cookie。

Secure。仅通过HTTPS传输Cookie。

SameSite。抵御CSRF攻击,LAX模式允许同一站点的跨站请求。

3.5 监控层——实时安全监测与响应

监控层通过日志记录和异常检测,及时发现并阻断攻击行为,这是分层架构中动态防护的关键环节。

3.5.1 认证日志系统

3.5.1.1 日志采集

定义类实现AuthenticationSuccessHandler接口,记录认证成功/失败事件、访问IP、时间戳等信息。

3.5.1.2 日志分析

使用ELK Stack(Elasticsearch、Logstash、Kibana)

进行日志存储与可视化,设置异常登录次数阈值(例如,5次/分钟),触发警报或自动封禁IP。

3.5.2 异常检测与攻击防护

(1) 暴力破解防护。通过UsernamePasswordAuthenticationFilter统计失败次数,超过阈值后锁定账户或IP。

(2) 会话固定攻击防护。用户登录成功后强制更换Session ID。

4 安全性分析与挑战

4.1 抵御常见攻击

常见攻击类型与对应防护措施如表2所示。

表2 常见攻击类型与对应防护措施

攻击类型	分层防护措施
密码嗅探	网络层HTTPS加密、认证层BCrypt哈希
CSRF	会话层SameSite Cookie、认证层Referer校验
JWT劫持	网络层HTTPS、会话层令牌短期有效+刷新机制
越权访问	授权层RBAC/ABAC、方法级安全注解
暴力破解	监控层登录频率限制、账户锁定

4.2 技术挑战与应对

(1) 性能优化。JWT签名验证和权限评估可能引入延迟,可通过缓存常用权限数据、使用硬件加速TLS加密缩短延迟时间。

(2) 令牌 revocation。无状态JWT难以主动失效,可引入黑名单机制或分布式缓存存储无效令牌。

(3) 多租户支持。在授权层设计租户隔离策略,通过用户属性(tenantId)限制资源访问范围。

5 结语

研究提出的Java生态Web应用用户鉴权立体防护架构,通过网络层、认证层、授权层、会话层、监控层的协同设计,构建了立体化的安全防护体系。结合Spring Security等主流框架,实现了从传输加密到业务逻辑权限控制的全链条安全保障,为Java Web应用提供更可靠的用户鉴权安全保障,推动各行各业数字化转型的安全落地。

参考文献

- [1] 蔡秋富. 基于威胁态势感知的校园网登录信息安全认证研究[J]. 电脑编程技巧与维护, 2025(3): 166-168.
- [2] 郑斑超. 多因素认证在网络安全中的应用与挑战[J]. 信息系统工程, 2024(12): 63-66.
- [3] 张小梅, 何菊, 余侃侃, 等. Django框架下的用户鉴权机制分析与研究[J]. 无线互联科技, 2023, 20(18): 146-148.

(下转第58页)



共执行 10 轮，取平均结果进行比较，指标包括语句覆盖率、分支覆盖率、平均测试时间和数据多样性。

表2 不同方法在测试对象T1上的测试性能对比

方法类型	语句覆盖率 (%)	分支覆盖率 (%)	平均测试时间 (s)	数据多样性 TD
RT	82.5	68.3	12.4	0.235
GA	91.2	80.4	21.7	0.264
RL	96.7	87.9	18.3	0.314

由表2可知，在测试对象T1（数学函数计算模块）上，提出的强化学习方法（RL）在各项指标上的表现均优于传统随机测试方法（RT）和遗传算法方法（GA）。在语句覆盖率方面，RL达到96.7%，相比RT的82.5%提升了14.2个百分点，相比GA的91.2%也提升了5.5个百分点；在分支覆盖率上，RL为87.9%，高于RT的68.3%和GA的80.4%，说明RL在挖掘深层条件路径方面具有更强的能力。这主要得益于强化学习中状态-动作-奖励机制的引入，能够根据反馈动态优化测试数据，持续探索未被覆盖的代码区域。在平均测试时间方面，RL为18.3 s，虽然长于RT的12.4 s，但优于GA的21.7 s，说明所提方法在保障测试质量的同时，兼顾了一定的执行效率。在数据多样性TD方面，RL达到0.314，远高于RT（0.235）和GA（0.264），说明生成的测试数据样本在变量空间分布上更具广度，避免了测试集“集中性”引发的路径冗余问题。

综合而言，所提方法在保持相对适中运行时间的前提下，实现了更高的覆盖率和更强的数据多样性，展现出其在复杂函数模块测试中的广泛适用性与优越性能。

4.3 测试用例覆盖率、效率和适应性评估结果

测试结果表明，所提方法在所有3个测试对象上的表现均优于对比方法，语句与分支覆盖率明显提高，生成的测试用例在数据多样性与缺陷发现能力方面表现更优，综合结果统计如表3所示。

表3 3种方法在全部测试对象上的平均评估指标

方法类型	平均语句覆盖率 (%)	平均分支覆盖率 (%)	平均测试时间 (s)	缺陷触发率 (%)	平均数据多样性 TD
RT	79.3	65.1	11.8	32.4	0.218
GA	89.6	77.5	20.9	41.3	0.256
RL	94.8	85.6	17.4	49.7	0.307

表3统计了3种测试方法在全部3个测试对象上的平均性能数据，从整体上进一步验证了所提方法的通用

性与有效性。在语句覆盖率方面，RL平均达到94.8%，相比GA高出5.2%、相比RT高出15.5%；在分支覆盖率上，RL同样以85.6%显著领先于GA（77.5%）和RT（65.1%），说明其能够更有效地捕捉程序的逻辑路径和条件分支。这种提升与RL策略中对“奖励函数”的持续优化密切相关，使得测试数据演化过程更具目标导向性。

在效率方面，RL平均测试时间为17.4 s，处于RT（11.8 s）与GA（20.9 s）之间，显示出良好的性能折中性。在缺陷触发率指标上，RL达到49.7%，远高于GA（41.3%）和RT（32.4%），表明其生成的测试用例更有可能覆盖潜在缺陷路径，提高了测试有效性。尤其值得关注的是数据多样性TD，RL仍保持最高值（0.307），表明其在多样本探索能力方面具备优势，有利于增强测试的覆盖全面性与鲁棒性。

表3的数据充分说明，强化学习方法不仅在单一模块上表现出色，在多模块测试中同样具备良好的稳定性和适应性，是一种具备广泛工程应用价值的测试用例生成手段。未来，在大规模测试系统中推广该方法具有显著的现实意义。

5 结语

提出了一种基于强化学习的自动化测试用例生成方法，通过构建包括程序分析、强化学习优化与适应性评价的模块化模型，实现测试数据的智能生成与迭代优化。所提方法在测试用例覆盖率、数据多样性与缺陷发现能力方面均显著优于传统方法。

参考文献

[1] 李可可. 基于遗传算法的自动化软件测试用例生成研究[J]. 电脑编程技巧与维护, 2024(12): 52-54.

[2] 张怀康, 姜林, 黄春营, 等. 基于元素树模型的GUI软件自动化测试用例生成[J]. 软件, 2024, 45(8): 88-91.

[3] 姬峰. 基于改进遗传算法的软件测试自动化机制研究[J]. 信息技术, 2019(10): 88-93.

[4] 周展鹏. 软件测试用例标签的自动化生成[J]. 电脑知识与技术, 2019, 15(8): 212-214.

[5] 赵鹤, 高婉玲. 基于模型的软件测试用例生成方法比较研究[J]. 现代计算机(专业版), 2017(4): 20-26.

[6] 苏兵. 基于Spring Security的权限管理系统的设计与实现[J]. 电脑与电信, 2023(9): 24-31.

(上接第55页)

[4] 顾慧杰. 基于微服务架构的用户认证与授权技术的研究与应用[D]. 北京: 北京印刷学院, 2022.

[5] 耿云霄, 许萌, 蔡军霞. Web应用常见单点登录方式